FabricSandbox：GPU 集群作业放置与故障重规划规划器

Option 4 — 自选项目 · 课程：数据通信网络模型与算法 · 组号：自定义3 · 成员：马贤纯

问题

GPU 训练集群在投入运行之前就要完成规划。放置不当会把集合通信流量挤上已经超订的上联链路，而一台交换机故障就可能同时拖慢许多作业。目前没有低成本的方式去回答“这个作业放在这里会怎样，或者那台 leaf 挂了会怎样”。

我们要构建什么

一个规划沙盘：给定 fabric 拓扑和一组训练作业，它产出放置方案、预测带宽行为，并在基础设施故障时重新规划。集群是模拟的；调度、路径计算和约束评估是真实执行的。

- **拓扑模型** —— spine/leaf/机柜/节点实体、链路容量、rail 对齐。
- **放置** —— 按作业声明的 GPU 数量与集合通信模式，做 rail 感知、碎片感知的调度。
- **影响分析** —— 瓶颈链路、超订比、吞吐代理指标、校验发现。
- **故障** —— 注入链路/交换机/worker 失效，重算可达性，给出迁移建议。
- **版本化** —— 草稿 → 校验 → 提交，支持修订版对比。

架构

> 浏览器（React/SVG）⇄ 网关（FastAPI）⇄ 消息总线（NATS）⇄ 放置服务 + N 个 pod worker ⇄ PostgreSQL

网关持有项目与修订版；放置服务派发候选分配方案。每个 worker 持有其 pod 的链路状态，在本地计算路径和负载，并与同伴交换链路状态更新。聚合器合并各项发现，通过 SSE 推送到界面。修订版不可变；变更在校验并提交之前始终是草稿。

核心优势（结构 • 行为 • 语义）

结构性。真正的多服务协作：网关、放置服务、聚合器和 pod worker 之间只通过网络通信；worker 各自持有不相交的 fabric 分区并交换链路状态事件，某个 worker 在评估中途消失时系统仍能给出结果。

行为性。决策而非固定流程：放置是在局部性、碎片化、余量之间做搜索；故障后的重规划依赖当前状态——可达性、在途作业、剩余容量；草稿到提交的生命周期是建立在不可变修订版之上的状态机，每一次变更都可追溯。

语义性。结果建立在真实的领域计算之上：由上联/下联容量推导超订比、跨 spine 切分的二分带宽、rail 对齐的集合通信流量，以及每个 leaf 的 GPU 与功耗预算下的可行性。

与 Option 2.3 的边界

我们把 fabric 拓扑的表示与通用网络设计校验视为已给定（属 Option 2.3 的范畴）。我们的贡献是拓扑之上发生的事情：作业放置、带宽感知的影响预测、故障重规划，以及修订版生命周期。

范围与评估

MVP：8 spine × 16 leaf × 8 节点 × 8 GPU（共 1024 GPU）、两种集合通信模式、rail 感知的贪心放置、两种故障场景（链路中断、worker 宕机）、修订版对比、脚本化演示。**扩展项：**基于 ILP 的放置、功耗预算、多租户配额。**不在范围内：**真实集群控制、实时遥测、热力建模。

验证：针对手工计算出的拓扑做单元测试；在运行中途杀掉一个 worker 的混沌测试，断言不丢失修订版且恢复时间有界；在相同工作负载上对比 rail 感知放置与朴素放置；一个六分钟演示（加载 → 调度 → 故障 → 重规划 → 对比 → 提交）。

里程碑与风险

W1 领域模型、拓扑生成器、界面骨架 · W2 放置与指标 · W3 worker、消息总线、故障注入 · W4 修订版与校验发现 · W5 混沌测试、演示 · W6 文档。

主要风险：范围蔓延到网络设计（已由上面的边界限定）、模拟保真度（假设需文档化并手工核对）、分布式调试成本（从第一天就做追踪）。